ORACLE & PostgreSQL

Bridging Oracle's Diagnostics Power with PostgreSQL's Native Performance Views

A technical side-by-side comparison of Oracle's Automatic Workload Repository and PostgreSQL's extensible monitoring ecosystem.

Kellyn Gorman
Engineer and Advocate, Redgate

Kellyn Gorman

Multiplatform and AI Advocate



Is This an Oracle vs. PostgreSQL Session?

I do love both, so who is this for?

- **If you're curious about the comparisons at this stage of the products.**
- PostgreSQL DBAs building production monitoring
- Platform engineers designing observability stacks
- Architects comparing database TCO
- Anyone who has said 'I miss AWR' after moving to Postgres

Oracle AWR: Architecture & Capabilities

SGA / PGA Instrumentation

~1,200+ wait events
Dynamic sampling

MMON/MMNL Background Process

Flushes dirty stats
Snapshot every 60 min

AWR Snapshots/ASH Samples (SYSAUX)

Configurable:
1–50,000 snapshots

Baselines & Compare Periods

Moving/fixed windows
Normalized diffs

What AWR Gives You

Time-Model Statistics

DB time breakdown: CPU, wait, parse, execute

Top Wait Events

Ranked waits by total seconds, counts, avg

SQL Statistics

Top SQL by elapsed, CPU, I/O, buffer gets

Historical Baselines

Fixed/moving windows for regression detection

Segment & I/O Stats

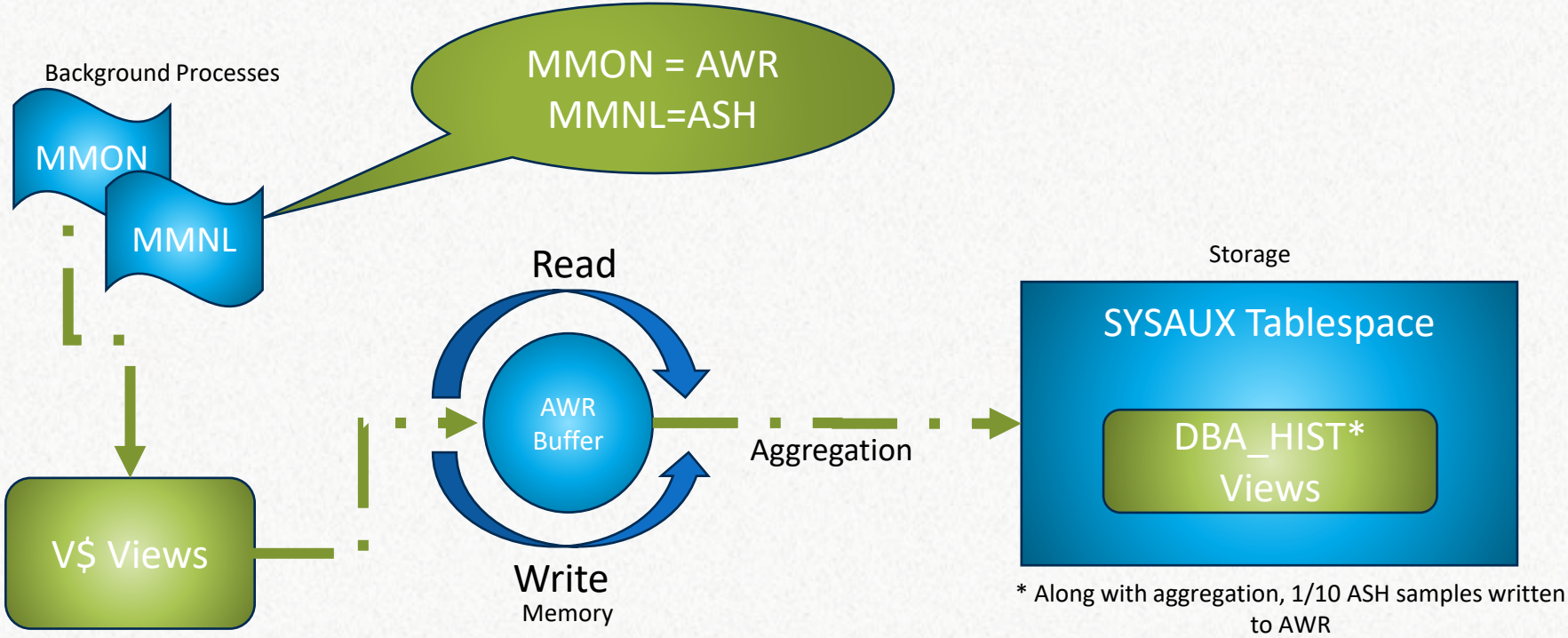
Hot segments, tablespace I/O, read/write latency

Latch & Mutex Detail

Contention breakdown to individual latch class

⚠ AWR requires the Oracle Diagnostics Pack license (~$7,800 per DB) — it is NOT included in Enterprise Edition by default

AWR Architecture





AWR Report Example

PostgreSQL Monitoring Primitives: Built-In & Extensions

Shared Buffers & Proc Mem

Buffer hits, WAL per-backend stats

Stats Collector Daemon

Writes .stat files per DB, table, fn

pg_stat_* Views (80+ catalogs)

In-memory, reset on restart/manual

Extensions & Schemas

Persistent history with pg_profile

PostgreSQL Monitoring Primitives: Built-In & Extensions

Core pg_stat_* Views

pg_stat_activity : Live sessions & queries

pg_stat_statements : Aggregated SQL stats

pg_stat_bgwriter : Checkpoint & buffer I/O

pg_stat_io : Reads, writes, evictions

pg_stat_wal : WAL generation rate

pg_stat_user_tables: Per-table tuple stats

pg_stat_user_indexes: Index usage & scans

pg_locks: Lock waits & holders

Extension Ecosystem

pg_stat_statements : SQL workload analysis

pg_profile : AWR-equivalent snapshots

pg_wait_sampling : Sampled wait event history

pg_buffercache : Buffer pool inspection

pg_store_plans : Plan history per query

pgBadger : Log-based report generation

Prometheus + exporter : Time-series metrics store

pganalyze : Commercial AWR-class SaaS

Feature Comparison — Oracle AWR vs PostgreSQL

Native Partial / Extension Gap / Manual Build Required

Capability	Oracle AWR	PostgreSQL	Delta
Snapshot-based history	✓ Native (AWR)	✓ pg_profile ext	Extension required
Configurable retention	✓ 1–50,000 snaps	✓ Configurable	Feature parity
Named baselines	✓ Fixed & moving	✗ Manual build	Custom SQL needed
Wait event detail (sampled)	✓ 1,200+ events	✓ pg_wait_sampling	Extension + sampling
SQL elapsed time ranking	✓ AWR Top SQL	✓ pg_stat_statements	No plan per exec
Per-execution plan capture	✓ AWR SQL Plan	✓ pg_store_plans	Extension required
Segment-level I/O stats	✓ v$segment_statistics	✓ pg_statio_*	Good parity
Buffer cache inspection	✓ vbh, xbh	✓ pg_buffercache	Extension required
Latch / mutex contention	✓ v$latch detail	△ Limited visibility	No latch equiv
Blocking / lock waits	✓ v$session history	✓ pg_locks + ASH-like	Needs sampling ext
Time-model statistics	✓ v$sess_time_model	✗ Not native	Custom build
Compare Periods report	✓ AWRDiff	✗ Manual SQL	No equivalent
OS / system metrics	✓ Included in AWR	△ External (Prometheus)	Agent required
Zero licensing cost	✗ Diagnostics Pack	✓ Fully open source	PG wins here

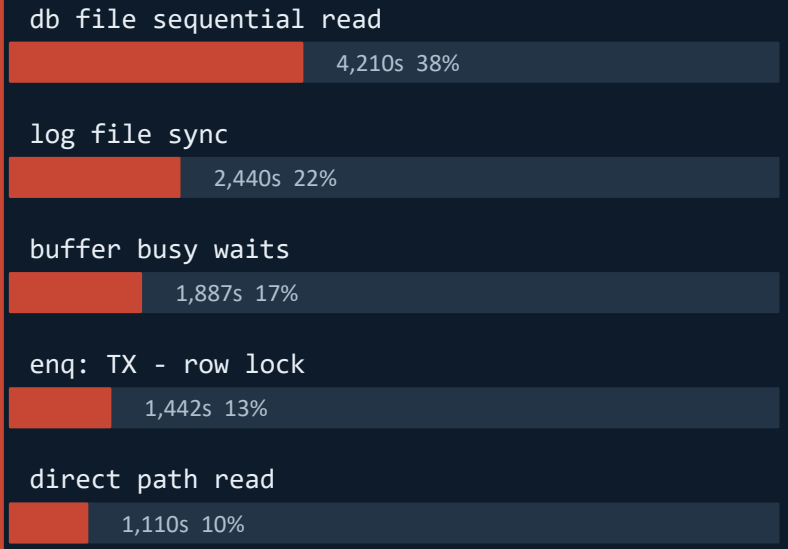
Querying Oracle and PostgreSQL Data



Wait Event Analysis — Oracle AWR vs PostgreSQL Sampling

Oracle

AWR Top 5 Wait Events (cumulative seconds)



PostgreSQL

pg_wait_sampling — sampled wait events (requires extension)

```
SELECT wait_event_type, wait_event,
       count(*)::numeric AS samples,
       round(count(*) * 100.0 /
         sum(count(*)) OVER (), 1) AS pct
FROM pg_wait_sampling_history
GROUP BY 1,2
ORDER BY 3 DESC
LIMIT 10;
```



Key Gap: Oracle samples waits 10x/sec natively; PostgreSQL requires pg_wait_sampling + bgworker..

SQL Ranking & Workload Analysis

Oracle: AWR Top SQL by Elapsed Time

```
SELECT sql_id,
  round(elapsed_time/1e6,1)  AS elapsed_sec,
  round(cpu_time/1e6,1)      AS cpu_sec,
  executions,
  round(elapsed_time/
    NULLIF(executions,0)/1e6,3)
                             AS avg_elapsed,
  buffer_gets,
  disk_reads,
  sql_text
FROM dba_hist_sqlstat s
JOIN dba_hist_sqltext t
  USING (sql_id)
WHERE snap_id BETWEEN :start AND :end
ORDER BY elapsed_time DESC
FETCH FIRST 10 ROWS ONLY;
```

PostgreSQL: pg_stat_statements Top SQL

```
SELECT
  left(query, 60)            AS query_snippet,
  calls,
  round(total_exec_time::numeric
        /1000, 2)            AS total_sec,
  round(mean_exec_time::numeric
        /1000, 4)            AS avg_sec,
  rows,
  shared_blks_hit,
  shared_blks_read,
  blk_read_time / 1000       AS read_sec,
  blk_write_time / 1000      AS write_sec
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

✓ **Both provide**

Total time, per-execution avg, I/O reads/writes, row counts

△ **PG advantage**

Query text normalized + hashed, no plan instability from literals

✗ **PG gap**

No per-snapshot history without pg_profile — stats reset on restart

✗ **PG gap**

No direct plan linkage without pg_store_plans extension

I/O Bottleneck Detection — pg_stat_io & pg_buffercache

pg_stat_io: PostgreSQL 16+ Native I/O Accounting

```
SELECT backend_type, object, context,
       reads, writes, extends,
       round(read_time::numeric/1000,2)  AS read_sec,
       round(write_time::numeric/1000,2) AS write_sec,
       hits, evictions, reuses,
       round(hits::numeric/(NULLIF(hits+reads,0))*100,1) AS hit_pct
FROM  pg_stat_io
ORDER BY reads DESC;
```

pg_buffercache: Hot Buffer Inspection (AWR Segment Equivalent)

```
SELECT  c.relname,
        count(*)                                    AS buffers_in_cache,
        round(count(*) * 8192 / 1024.0 / 1024, 1) AS size_mb,
        round(100.0 * count(*) /
              (SELECT count(*) FROM pg_buffercache), 1) AS pct_of_cache,
        count(*) FILTER (WHERE b.isdirty)           AS dirty_buffers
FROM    pg_buffercache b
JOIN    pg_class c ON c.relfilenode = b.relfilenode
GROUP BY 1 ORDER BY 2 DESC LIMIT 15;
```

The PostgreSQL AWR-Equivalent Extension Stack

pg_stat_statements

Core (built-in)



Top SQL by elapsed, I/O, rows

```
CREATE EXTENSION pg_stat_statements;
```

pg_profile

Production-ready



AWR snapshots, baselines, HTML reports

```
CREATE EXTENSION pg_profile;
```

pg_wait_sampling

Production-ready



ASH / wait event history

```
shared_preload_libraries = 'pg_wait_sampling'
```

pg_buffercache

Core (contrib)



Buffer cache / segment analysis

```
CREATE EXTENSION pg_buffercache;
```

pg_store_plans

Community extension



Execution plan history per query

```
shared_preload_libraries = 'pg_store_plans'
```

auto_explain

Core (contrib)



Log slow query plans (like Oracle event 10053)

```
shared_preload_libraries = 'auto_explain'
```

pg_profile

We can get part of the way there…



Building AWR-Style Snapshots in PostgreSQL

1 Install pg_profile

```
CREATE EXTENSION
pg_profile;
– Creates snapshot
infrastructure
– Configures
retention (default
7 days)
– Sets up
background snapshot
worker
```

2 Take Snapshots

```
SELECT
snapshot('main');
– Manual or pg_cron
scheduled
– Captures
pg_stat_* delta
– Stores in profile
schema tables
```

3 Generate Report

```
SELECT get_report(
  'main', 1, 5
);
– HTML or text
output
– Top SQL, waits,
I/O, growth
```

Example pg_profile Snapshot Report

In Addition to pg_profile?

Without pg_profile



I/O Bottleneck Detection — pg_stat_io & pg_buffercache

- pg_stat_io (PG16+): reads, writes, hit rate, evictions per backend type — closest Oracle equivalent to v$filestat
- pg_buffercache identifies which relations hold the most buffer pages — same insight as Oracle's v$bh / buffer cache segments
- pg_stat_io counters reset on pg_stat_reset() and on server restart — snapshot before clearing

What is I want historical data, even after a reset?

Building AWR-Style Snapshots in PostgreSQL

Custom Snapshot Framework (without pg_profile)

```
-- Minimal snapshot table
CREATE TABLE perf_snapshots AS
  SELECT now() AS snap_time, queryid, query, calls, total_exec_time,
         shared_blks_hit, shared_blks_read, rows
  FROM pg_stat_statements WHERE false;

-- Capture procedure (run via pg_cron every 30 min)
INSERT INTO perf_snapshots
  SELECT now(), queryid, query, calls, total_exec_time,
         shared_blks_hit, shared_blks_read, rows
  FROM pg_stat_statements;

-- Delta report between two snapshots
SELECT query, (s2.calls - s1.calls) AS new_calls,
       round((s2.total_exec_time - s1.total_exec_time)/1000, 2) AS added_secs
FROM perf_snapshots s1 JOIN perf_snapshots s2 USING (queryid)
WHERE s1.snap_time = :'start_ts' AND s2.snap_time = :'end_ts'
ORDER BY added_secs DESC LIMIT 10;
```

Example of a Manually Generated AWR Style Report in PostgreSQL

How About ASH?

Blocking & Concurrency Pattern Detection

Active Session & Blocking Tree

```
WITH blocking AS (
  SELECT pid, usename, datname,
         wait_event_type, wait_event,
         state, query_start,
         pg_blocking_pids(pid) AS blockers,
         left(query,80) AS query
  FROM pg_stat_activity
  WHERE state != 'idle'
)
SELECT
  pid, state, wait_event,
  array_length(blockers,1) AS blocked_by_n,
  blockers,
  now()-query_start AS duration,
  query
FROM blocking
ORDER BY blocked_by_n DESC NULLS LAST;
```

Long-Running Transactions & Idle-in-Transaction

```
-- Sessions idle-in-transaction
-- (danger: holding locks, blocking vacuums)
SELECT pid, usename, datname,
       state,
       now() - xact_start AS txn_age,
       now() - state_change AS idle_since,
       left(query, 100) AS last_query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
  AND xact_start < now()
          - interval '5 minutes'
ORDER BY txn_age DESC;

-- Lock detail for a specific pid
SELECT locktype, relation::regclass,
       mode, granted
FROM pg_locks
WHERE pid = :target_pid;
```

```
       SID STATUS   EVENT                                BLOCKED_BY BLOCKING_SE
SECONDS_IN_WAIT
---------- -------- ---------------------------------- ---------- ----------- -----
----------
QUERY
--------------------------------------------------------------------------------

       274 ACTIVE   enq: TX - row lock contention            341 VALID
2847
UPDATE orders SET status = 'PROCESSING' WHERE order_id = 8841204

       419 ACTIVE   enq: TX - row lock contention              341 VALID
1203
UPDATE inventory SET reserved = reserved +
```

```
   locktype    |      relation      |      mode       | granted
---------------+--------------------+-----------------+---------
 relation      | orders             | RowExclusiveLock| t
 relation      | orders_pkey        | RowExclusiveLock| t
 relation      | orders_user_id_idx | RowExclusiveLock| t
 tuple         | orders             | ExclusiveLock   | t
 transactionid |                    | ExclusiveLock   | t
 virtualxid    |                    | ExclusiveLock   | t
(6 rows)
```

Blocking & Concurrency Pattern Detection

`idle in transaction`

Holds row locks; blocks VACUUM; can cause table bloat. Alert if age > 5 min.

`pg_blocking_pids()`

Returns PIDs of all sessions blocking a given PID. Recursive blocker chains are visible.

`Oracle ASH Gap`

No native Active Session History in PG. Use pg_wait_sampling + periodic pg_stat_activity snapshots.

Planning Performance Collection

Takeaways & Your Monitoring Roadmap

What you can do today to achieve enterprise-grade PostgreSQL diagnostics

Day 1 — Baseline

- Enable pg_stat_statements
- Enable track_io_timing in postgresql.conf
- Verify pg_stat_io available (PG 16+)
- Baseline query with pg_stat_bgwriter

Week 1 — History

- Install pg_profile
- Schedule snapshot every 30-60 min via pg_cron
- Tune retention to 14+ days
- Review first AWR-style HTML report

Takeaways & Your Monitoring Roadmap

What you can do today to achieve enterprise-grade PostgreSQL diagnostics

Month 1 — Deep Diag

- Add pg_wait_sampling for wait history
- Add pg_buffercache for cache inspection
- Add pg_store_plans for plan history
- Build alerting on blocking sessions

Quarter — Baselines

- Establish peak-load snapshot baselines
- Build compare-period delta reports
- Integrate with Prometheus/Grafana
- Document tuning runbooks

Takeaways & Your Monitoring Roadmap

What you can do today to achieve enterprise-grade PostgreSQL diagnostics

The Bottom Line

- pg_stat_* covers ~70% of AWR use cases natively, out of the box
- pg_profile is the closest open-source AWR equivalent available
- Biggest gap: no native ASH: use pg_wait_sampling as substitute
- Stack is open-source, cloud-portable: no Diagnostics Pack required

Questions?

Resources to Take With You



pg_profile

github.com/zubkov-andrei/pg_profile



pg_wait_sampling

github.com/postgrespro/pg_wait_sampling



pg_store_plans

github.com/ossc-db/pg_store_plans



pg_stat_statements

postgresql.org/docs/current/pgstatstatements.html



pg_buffercache

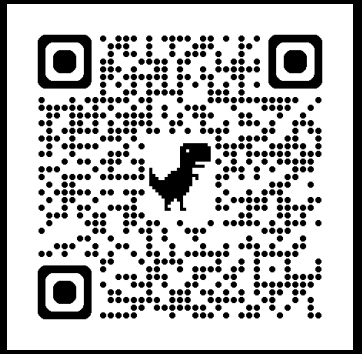
postgresql.org/docs/current/pgbuffercache.html



Official PG Stats Ref

postgresql.org/docs/current/monitoring-stats.html

Thank you!



Kellyn Gorman
Engineer and Advocate, Redgate